

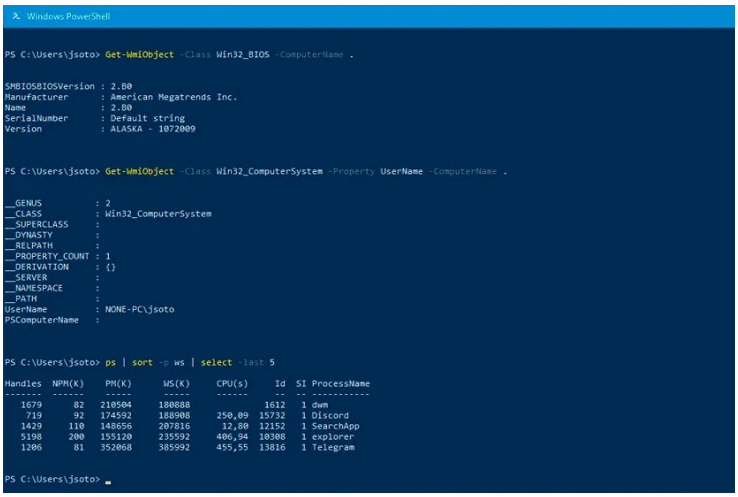
Administración de Sistemas Operativos

UD – 1

Script PowerShell

Contenidos





ÍNDICE

1. ¿QUÉ ES POWERSHELL? **3**

Versiones 3

Diferencias prácticas 5.1 vs 7+ que verás en este manual: *3*

¿POR QUÉ APRENDER SCRIPTING POWERSHELL EN ASIR? **3**

ESTRUCTURA BÁSICA DE UN SCRIPT **3**

Variables automáticas comunes en administración *4*

Variables de usuario *4*

Aritmética en PowerShell *5*

Parámetros de script (param()) *5*

Parámetros especiales *6*

Comillas: simples, dobles y escape *6*

REDIRECCIONAMIENTO Y TUBERÍAS **7**

Redireccionamiento *7*

Tuberías (¡objetos, no texto!) *7*

ESTRUCTURAS CONDICIONALES **8**

Operadores de comparación *8*

Operadores lógicos *8*

Estructura if *9*

Estructura switch *9*

ESTRUCTURAS DE REPETICIÓN (BUCLES) **10**

Bucle foreach *10*

Bucle while y do-while *10*

Control de bucles *11*

FUNCIONES **11**

Definición y llamada *11*

Parámetros *11*

Retorno de valores *12*

POWERSHELL APLICADO A LA ADMINISTRACIÓN (AMPLIACIÓN ASIR) **12**

Gestión de procesos y servicios **12**

Gestión de usuarios locales *13*

Active Directory con PowerShell *13*

Inventario masivo desde CSV *14*

PLANTILLA DE SCRIPT PROFESIONAL **15**

Descripción o cabecera del script *15*

Funciones *15*

Zona de Validación *16*

Principal *16*

Tabla de equivalencias rápida Bash ↔ PowerShell **17**

1. ¿QUÉ ES POWERSHELL?

PowerShell es una **solución de automatización de tareas multiplataforma** formada por un shell de línea de comandos, un lenguaje de scripting y un marco de configuración. Originalmente diseñado para Windows, PowerShell 7+ funciona también en Linux y macOS.

A diferencia de Bash (y de CMD), PowerShell **trabaja con objetos .NET**, no con texto plano. Esto significa que cuando ejecutas Get-Process, no recibes una tabla de texto: recibes objetos System.Diagnostics.Process con propiedades y métodos a los que puedes acceder directamente. Esta es **la diferencia más importante** respecto a Bash.

PowerShell es tanto un intérprete de comandos (ejecuta órdenes escritas en consola) como un lenguaje de programación completo (variables, condicionales, bucles, funciones, clases).

Su sintaxis sigue el patrón **Verbo-Sustantivo**: **CMD - LET**

Get-Service, Set-Item, New-User, Remove-Item.



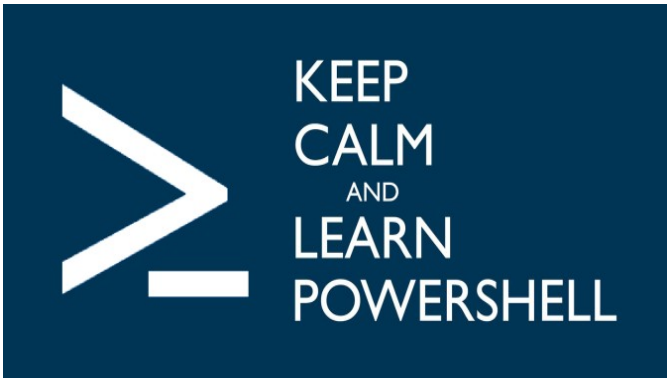
Este manual está basado en **Windows PowerShell 5.1**, la versión que viene instalada por defecto en Windows Server 2016/2019 y Windows 10/11, y que se encuentra en la gran mayoría de entornos empresariales con dominio Active Directory . Se indicarán las diferencias relevantes con PowerShell 7+ (multiplataforma) donde aplique, ya que es la versión en evolución y útil si continúas con administración cloud (Azure, Microsoft 365).

$PSVersionTable

1. ¿POR QUÉ APRENDER SCRIPTING POWERSHELL EN ASIR?

En el módulo de Administración de Sistemas Operativos (y en general en todo ASIR), PowerShell es una competencia fundamental en entornos Windows porque permite:

- **Automatizar tareas repetitivas** → creación masiva de usuarios en Active Directory, copias de seguridad, monitorización de servicios, limpieza de logs.
- **Administrar Active Directory** → crear, modificar y eliminar usuarios, grupos y equipos de forma masiva desde CSV.
- **Gestionar servicios del sistema** → reiniciar servicios caídos, consultar logs de eventos, programar tareas.
- **Administración remota** → ejecutar comandos y scripts en múltiples servidores simultáneamente mediante Invoke-Command y WinRM.
- **Integrar herramientas** → combinar cmdlets del sistema con módulos de Hyper-V, DNS, DHCP, Azure, Microsoft 365.
- **Documentar procedimientos** → los scripts .ps1 sirven como documentación ejecutable de procesos administrativos.



2. ESTRUCTURA BÁSICA DE UN SCRIPT

Todo script PowerShell se guarda con extensión .ps1. No existe una línea "shebang" obligatoria como en Bash, aunque en PowerShell 7+ (Linux) se puede usar #!/usr/bin/env pwsh. El script se ejecuta:

```
# Crear el script con cualquier editor de texto (VS Code, Notepad++, ISE)
# Guardarlo como mi_script.ps1

# Ejecutarlo desde consola:
.\mi_script.ps1

# Si da error de política de ejecución:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Los comentarios en PowerShell comienzan con # para una línea, o se encierran entre <# ... #> para bloques.

```
# Este es un comentario de una línea

<#
  Este es un comentario
  de múltiples líneas
#>
```

2.1 Variables automáticas comunes en administración

PowerShell expone variables automáticas (también llamadas "preference variables" o variables del sistema) que contienen información del entorno:

$env:COMPUTERNAME → *Nombre del equipo*
$env:USERNAME → *Nombre del usuario actual*
$env:PATH → *Rutas de búsqueda de ejecutables*
$PWD → *Directorio de trabajo actual (objeto, no string)*
$PSVersionTable → *Versión de PowerShell y detalles*
**$?** → *$true si el último comando tuvo éxito, $false si no*
$Error → *Array con los últimos errores*
$HOME → *Directorio personal del usuario*

Para acceder a variables de entorno se usa $env:NOMBRE. Para listarlas todas: Get-ChildItem Env:.

2.2 Variables de usuario

Las variables en PowerShell se declaran con $ delante del nombre. A diferencia de Bash, **no hay restricción de espacios** alrededor del = (aunque por estilo se recomienda no ponerlos).

```
$SERVIDOR = "192.168.1.10"
$PUERTO = 22
$USUARIO = "administrador"

Write-Host "Conectando a $SERVIDOR en el puerto $PUERTO como $USUARIO"
```

PowerShell **infiere el tipo** de la variable según el valor asignado. Para forzar un tipo:

```
[int]$puerto = 22
[string]$servidor = "192.168.1.10"
```

2.3 Aritmética en PowerShell

Sintaxis	Descripción
`$a + $b`	Suma
`$a - $b`	Resta
`$a * $b`	Multiplicación
`$a / $b`	División
`$a % $b`	Módulo (resto)
`$a++`	Incremento en 1
`$a += 5`	Incremento en 5

2.4 Parámetros de script (param())

Los scripts pueden recibir argumentos desde la línea de comandos. En PowerShell se declaran al principio del script con el bloque param():

```
# Al principio del script (después de comentarios y #Requires)
param(
    [Parameter(Mandatory=$true)]
    [string]$Nombre,

    [Parameter(Mandatory=$false)]
    [string]$Grupo = "Usuarios"
)

Write-Host "Creando usuario $Nombre en grupo $Grupo"
```

Parámetros posicionales vs nombrados:

```
# Llamada con parámetros nombrados (recomendada, más legible)
.\script.ps1 -Nombre "ana" -Grupo "Administradores"

# Llamada posicional (según el orden en param())
.\script.ps1 "ana" "Administradores"
```

Se pueden añadir **validaciones** directamente en el param():

```
param(
    [ValidateSet("crear","eliminar","listar")]
    [string]$Accion = "listar",

    [ValidateNotNullOrEmpty()]
    [string]$Usuario
)
```

2.5 Parámetros especiales

Variable	Significado
`$args`	Array con argumentos no declarados en `param()`
`$PSBoundParameters`	Hashtable con los parámetros que se pasaron realmente al script/función
`$MyInvocation`	Información sobre cómo se invocó el script
`$?`	`$true` si el último comando tuvo éxito

2.6 Comillas: simples, dobles y escape

En PowerShell, la diferencia es la misma que en Bash:

```
Write-Host '$HOME'    # Muestra: $HOME
```

•**Comillas simples** '...': todo es **literal**, no se expanden variables.

```
Write-Host "$HOME"    # Muestra: C:\Users\usuario (o /home/usuario en PS7 Linux)
```

•**Comillas dobles** "...": se **expanden variables** y caracteres de escape.

```
Write-Host "El precio es `$100"   # Muestra: El precio es $100 (sin expandir)
Write-Host "Línea 1`nLínea 2"     # `n = salto de línea
```

•**Carácter de escape** en PowerShell es el **backtick** ` (no la barra invertida \ como en Bash).

3. REDIRECCIONAMIENTO Y TUBERÍAS

3.1 Redireccionamiento

PowerShell maneja **múltiples flujos de salida** (streams). Los principales:

Stream	Descripción
1	**Success** (salida normal)
2	**Error**
3	**Warning**
4	**Verbose**
5	**Debug**

Operador	Significado	Ejemplo
`>`	Redirige success (sobrescribe)	`Get-Process > procesos.txt`
`>>`	Redirige success (añade)	`Get-Date >> registro.txt`
`2>`	Redirige error	`comando 2> errores.txt`
`2>&1`	Redirige error a success	`comando 2>&1 > salida.txt`
`*>`	Redirige todos los streams	`script.ps1 *> todo.txt`

3.2 Tuberías (¡objetos, no texto!)

La tubería | en PowerShell **pasa objetos entre comandos**, no texto. Esto es radicalmente distinto a Bash.

```
# POWERSHELL: Get-Process devuelve OBJETOS, Where-Object filtra por PROPIEDADES
Get-Process | Where-Object { $_.ProcessName -like "*apache*" } | Measure-Object
```

Implicaciones prácticas:

- No necesitas awk/sed/cut para extraer campos: accedes directamente a propiedades ($_.Name, $_.Id, $_.CPU).
- Puedes ordenar por cualquier propiedad: Get-Process | Sort-Object CPU -Descending
- Puedes seleccionar columnas específicas: Get-Process | Select-Object Name, Id, CPU
- Puedes exportar estructuras: Get-Process | Export-Csv -Path procesos.csv -NoTypeInformation

Los cmdlets más usados en tuberías son:

- **Where-Object** (alias ?): filtra objetos según condición.
- **Select-Object** (alias select): elige propiedades o primeras N.

- **Sort-Object** (alias sort): ordena.
- **ForEach-Object** (alias %): ejecuta acción por cada objeto.
- **Measure-Object**: cuenta, suma, promedio.

```
# Los 10 procesos que más CPU consumen
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 Name, CPU

# Servicios detenidos y su nombre para mostrar
Get-Service | Where-Object { $_.Status -eq "Stopped" } | Select-Object Name, DisplayName
```

4. ESTRUCTURAS CONDICIONALES

4.1 Operadores de comparación

PowerShell usa operadores con guion (-eq, -gt, etc.) en lugar de ==, >, etc. -6:

Operador	Significado
`-eq`	Igual a
`-ne`	Distinto de
`-gt`	Mayor que
`-ge`	Mayor o igual
`-lt`	Menor que
`-le`	Menor o igual
`-like`	Coincidencia con wildcards (`*` , `?`)
`-notlike`	No coincide con wildcard
`-match`	Coincidencia con expresión regular
`-contains`	El array contiene un valor

4.2 Operadores lógicos

Operador	Significado	Ejemplo
`-and`	Y lógico	`($a -gt 0) -and ($b -gt 0)`
`-or`	O lógico	`($a -eq 1) -or ($b -eq 2)`
`-not` o `!`	Negación	`-not (Test-Path $archivo)`
`-xor`	O exclusivo	`($a -eq 1) -xor ($b -eq 1)`

4.3 Estructura if

```
if (Test-Path "/etc/passwd") {
    Write-Host "El archivo existe"
} else {
    Write-Host "El archivo NO existe"
}
```

elseif (todo junto, sin guion):

```
if ($1 -eq "start") {
    Write-Host "Iniciando servicio..."
} elseif ($1 -eq "stop") {
    Write-Host "Deteniendo servicio..."
} else {
    Write-Host "Uso: $0 {start|stop}"
    exit 1
}
```

4.4 Estructura switch

switch es la alternativa a múltiples if-elseif. Es más limpio y permite patrones con wildcards y regex:

```
$opcion = Read-Host "Introduce el número de tu opción"

switch ($opcion) {
    "1" {
        Write-Host "Iniciando copia de seguridad..."
        # comando de respaldo
    }
    "2" {
        Write-Host "Buscando actualizaciones..."
        # winget upgrade --all
    }
    "3" {
        Write-Host "Saliendo del programa. ¡Adiós!"
        exit 0
    }
    default {
        Write-Host "Opción no válida. Por favor, elige 1, 2 o 3."
    }
}
```

5. ESTRUCTURAS DE REPETICIÓN (BUCLES)

5.1 Bucle foreach

El bucle foreach itera sobre una colección de objetos. Es **el más usado en administración de sistemas** porque PowerShell devuelve objetos (procesos, servicios, usuarios) que necesitas recorrer:

```
# Iterar sobre archivos de log
foreach ($archivo in Get-ChildItem "C:\Logs\*.log") {
    Write-Host "Procesando: $($archivo.Name)"
    # Compress-Archive -Path $archivo.FullName -DestinationPath "$($archivo.FullName).zip"
}
```

Sintaxis alternativa con tubería (más idiomática en PowerShell):

```
# Iterar sobre una lista de nombres
foreach ($usuario in @("ana", "luis", "pedro", "maria")) {
    Write-Host "Creando usuario: $usuario"
    # New-LocalUser -Name $usuario -NoPassword
}
```

Nota: foreach (keyword) y ForEach-Object (cmdlet) son similares pero no idénticos. foreach carga toda la colección en memoria; ForEach-Object procesa objeto a objeto en streaming.

5.2 Bucle while y do-while

Ejecuta mientras la condición sea verdadera:

```
$contador = 1
while ($contador -le 5) {
    Write-Host "Iteración $contador"
    $contador++
}
```

do-while ejecuta **al menos una vez** antes de evaluar la condición:

```
$respuesta = ""
do {
    $respuesta = Read-Host "¿Deseas continuar? (s/n)"
} while ($respuesta -ne "s" -and $respuesta -ne "n")
```

5.3 Control de bucles

- **break**: sale del bucle inmediatamente.
- **continue**: salta a la siguiente iteración.

```
foreach ($num in 1..10) {
    if ($num -eq 5) {
        continue   # Salta el 5
    }
    if ($num -eq 8) {
        break      # Se detiene al llegar a 8
    }
    Write-Host $num
}
```

6. FUNCIONES

Las funciones permiten organizar y reutilizar código.

6.1 Definición y llamada

```
# Definición
function Saludar {
    param([string]$Nombre)
    Write-Host "Hola, $Nombre"
}

# Llamada
Saludar -Nombre "Mundo"
```

6.2 Parámetros

```
function Crear-Usuario {
    param(
        [Parameter(Mandatory=$true)]
        [string]$Nombre,

        [Parameter(Mandatory=$false)]
        [string]$Grupo = "Usuarios"
    )

    Write-Host "Creando $Nombre en grupo $Grupo"
    # New-LocalUser -Name $Nombre
    # Add-LocalGroupMember -Group $Grupo -Member $Nombre
}
```

Las funciones reciben argumentos mediante param() dentro de la función. Se pueden usar los mismos atributos que en scripts (Mandatory, ValidateSet, etc.):

6.3 Retorno de valores

PowerShell no usa return como en otros lenguajes para "imprimir" un valor. **Todo lo que se emite en la función se devuelve** (se añade al pipeline). Se puede usar return para salir anticipadamente:

```powershell
function Contar-Archivos {
    param([string]$Directorio)

    $total = (Get-ChildItem $Directorio -File).Count
    return $total   # Emite el valor y sale
}

$total = Contar-Archivos -Directorio "C:\Windows"
Write-Host "Hay $total archivos en C:\Windows"
```

7. POWERSHELL APLICADO A LA ADMINISTRACIÓN (AMPLIACIÓN ASIR)

Esta sección no existe como tal en el manual de Bash, pero es **imprescindible para ASIR** porque PowerShell es la herramienta principal de administración Windows.

7.1 Gestión de procesos y servicios

```powershell
# Listar procesos ordenados por uso de memoria
Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 10 Name, Id, @{Name="RAM(MB)";Expres
sion={[math]::Round($_.WorkingSet64/1MB,2)}}

# Detener un proceso por nombre
Stop-Process -Name "notepad" -Force

# Servicios: listar detenidos
Get-Service | Where-Object Status -eq "Stopped"

# Reiniciar un servicio
Restart-Service -Name "Spooler"

# Iniciar/detener servicio
Start-Service -Name "DHCP"
Stop-Service -Name "DHCP" -Force
```

7.2 Gestión de usuarios locales

En Windows Server con PowerShell 5.1, el módulo LocalAccounts está disponible :

```powershell
# Listar usuarios locales
Get-LocalUser

# Crear un usuario local
New-LocalUser -Name "ana" -Password (ConvertTo-SecureString "P@ssw0rd" -AsPlainText -Force)
]
# Añadir a grupo
Add-LocalGroupMember -Group "Administradores" -Member "ana"

# Eliminar usuario
Remove-LocalUser -Name "ana"
```

7.3 Active Directory con PowerShell

El módulo ActiveDirectory se instala automáticamente al añadir el rol AD DS en Windows Server, o manualmente con RSAT en estaciones de trabajo

Información de dominio y controladores:

```powershell
# Listar todos los controladores de dominio
Get-ADDomainController -Filter * | Format-Table Hostname, Site

# Ver información del dominio
Get-ADDomain
```

Cmdlets básicos de AD:

```powershell
# Importar el módulo
Import-Module ActiveDirectory

# Listar usuarios
Get-ADUser -Filter *

# Buscar usuario por nombre
Get-ADUser -Filter { Name -like "*ana*" }

# Obtener propiedades completas de un usuario
Get-ADUser -Identity "ana" -Properties *

# Crear usuario en una OU específica
New-ADUser -Name "Ana García" `
           -SamAccountName "agarcia" `
           -UserPrincipalName "agarcia@contoso.com" `
           -Path "OU=Usuarios,DC=contoso,DC=com" `
           -AccountPassword (ConvertTo-SecureString "P@ssw0rd" -AsPlainText -Force) `
           -Enabled $true

# Añadir usuario a grupo
Add-ADGroupMember -Identity "Administradores del dominio" -Members "agarcia"

# Deshabilitar usuario
Disable-ADUser -Identity "agarcia"

# Listar equipos del dominio
Get-ADComputer -Filter * | Select-Object Name, OperatingSystem
```

7.4 Inventario masivo desde CSV

Caso práctico muy habitual en ASIR: crear usuarios en AD desde un archivo CSV.

Archivo usuarios.csv:

```csv
Nombre,SamAccountName,OU
Ana García,agarcia,OU=Usuarios,DC=contoso,DC=com
Luis Pérez,lperez,OU=Usuarios,DC=contoso,DC=com
María López,mlopez,OU=Usuarios,DC=contoso,DC=com
```

Script crear_usuarios.ps1:

```powershell
param(
    [Parameter(Mandatory=$true)]
    [string]$RutaCSV
)

Import-Module ActiveDirectory

$usuarios = Import-Csv -Path $RutaCSV -Delimiter ","

foreach ($usuario in $usuarios) {
    $password = ConvertTo-SecureString "P@ssw0rd" -AsPlainText -Force

    New-ADUser -Name $usuario.Nombre `
               -SamAccountName $usuario.SamAccountName `
               -UserPrincipalName "$($usuario.SamAccountName)@contoso.com" `
               -Path $usuario.OU `
               -AccountPassword $password `
               -Enabled $true

    Write-Host "Usuario $($usuario.SamAccountName) creado correctamente"
}
```

Ejecución:

```powershell
.\crear_usuarios.ps1 -RutaCSV "C:\temp\usuarios.csv"
```

8. PLANTILLA DE SCRIPT PROFESIONAL

Con el objetivo de mantener la misma filosofía de trabajo entre todos los alumnos del grupo, se utilizará la siguiente plantilla genérica para todos los scripts:

8.1 Descripción o cabecera del script

```
<#
.SYNOPSIS
    Script para gestión de usuarios en Active Directory

.DESCRIPTION
    Crea usuarios en AD desde un archivo CSV con formato:
    Nombre,SamAccountName,OU

.PARAMETER RutaCSV
    Ruta completa al archivo CSV de entrada

.EXAMPLE
    .\gestion_usuarios.ps1 -RutaCSV "C:\temp\usuarios.csv"

.NOTES
    Autor: Alumno ASIR
    Fecha: 2025-01-15
    Versión PowerShell: 5.1
#>
```

8.2 Funciones

```
# --- Funciones ---

function Mostrar-Uso {
    Write-Host "Uso: .\gestion_usuarios.ps1 -RutaCSV <ruta_csv>"
    Write-Host "Ejemplo: .\gestion_usuarios.ps1 -RutaCSV 'C:\temp\usuarios.csv'"
    exit 1
}
```

8.3 Zona de Validación

```
# --- Validación ---

param(
    [Parameter(Mandatory=$false)]
    [string]$RutaCSV
)
```

8.4 Principal

```powershell
# --- Principal ---

$usuarios = Import-Csv -Path $RutaCSV

foreach ($usuario in $usuarios) {
    try {
        New-ADUser -Name $usuario.Nombre `
                   -SamAccountName $usuario.SamAccountName `
                   -UserPrincipalName "$($usuario.SamAccountName)@contoso.com" `
                   -Path $usuario.OU `
                   -AccountPassword (ConvertTo-SecureString "P@ssw0rd" -AsPlainText -Force) `
                   -Enabled $true

        Write-Host "[OK] Usuario $($usuario.SamAccountName) creado correctamente" -ForegroundColor Green
    }
```

9. TABLA DE EQUIVALENCIAS RÁPIDA BASH ↔ POWERSHELL

#!/bin/bash	→	*No requerido (opcional #Requires -Version 5.1)*
echo	→	*Write-Host / Write-Output*
$VAR	→	*$VAR*
'literal'	→	*'literal'*
"expande $VAR"	→	*"expande $VAR"*
\ (escape)	→	*` (backtick) |*
| (pipes de texto)	→	*| (pipes de objetos)*
grep	→	*Where-Object / Select-String*
awk / cut	→	*Select-Object*
sed	→	*-replace / ForEach-Object*
if [...]; then	→	*if (...) { ... }*
case	→	*switch*
for x in ...; do	→	*foreach ($x in ...) { }*
while [...]; do	→	*while (...) { }*
function nombre() {}	→	*function Nombre {}*
$1, $2	→	*param($a, $b)*
$?	→	*$? ($true/$false)*
chmod +x	→	*Set-ExecutionPolicy*
2>	→	*2> (pero con streams)*